OntarioTech Business & IT

Applied Cryptography

Winter 2026

Random Number Generators (RNGs)

- Three main categories
  1. True Random Number Generator (TRNG)
     - Based on a physical random process: coin flipping, radioactive decay, mouse movement
     - Given a sequence of n output, cannot predict $n+1$ with certainty
  2. Pseudorandom Number Generator (PRNG)
     - Generates the sequence from an initial seed value
     - typically, the output stream has good statistical properties (e.g., uniformity)

$$s_0 = seed$$
$$s_{i+1} = f(s_i, s_{i-1}, \dots, s_{i-t})$$

     - Most PRNGs have **bad** cryptographic properties
  3. Cryptographically Secure Random Number Generator (SRNG)
     - Special PRNG when output is unpredictable, i.e., given n consecutive bits of output s_i

Random Number Generators (RNGs) – cont'd

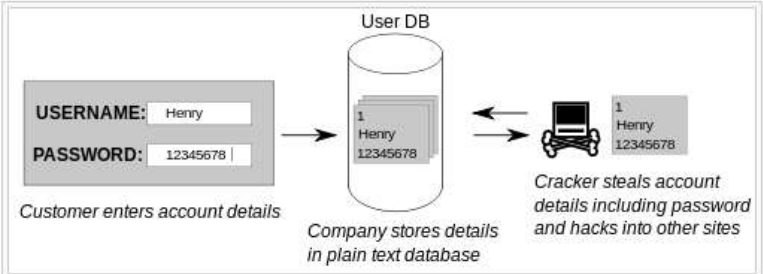
- Using SRNG (when random numbers are needed) is imperative to the security of your cryptographic scheme used in your application.

```
>>> import secrets
>>> secrets.token_urlsafe(16)
'zs9XYCbTPKvux46UJckflw'
>>> secrets.token_hex(16)
'6bef18936ac12a9096e9fe7a8fe1f777'
```

```
>>> from random import SystemRandom
>>> cryptogen = SystemRandom()
>>> [cryptogen.randrange(3) for i in range(20)] # random ints in range(3)
[2, 2, 2, 2, 1, 2, 1, 2, 1, 0, 0, 1, 1, 0, 0, 2, 0, 0, 0, 0]
>>> [cryptogen.random() for i in range(3)]  # random floats in [0., 1.)
[0.2710009745425236, 0.016722063038868695, 0.8207742461236148]
```

Password Hashing

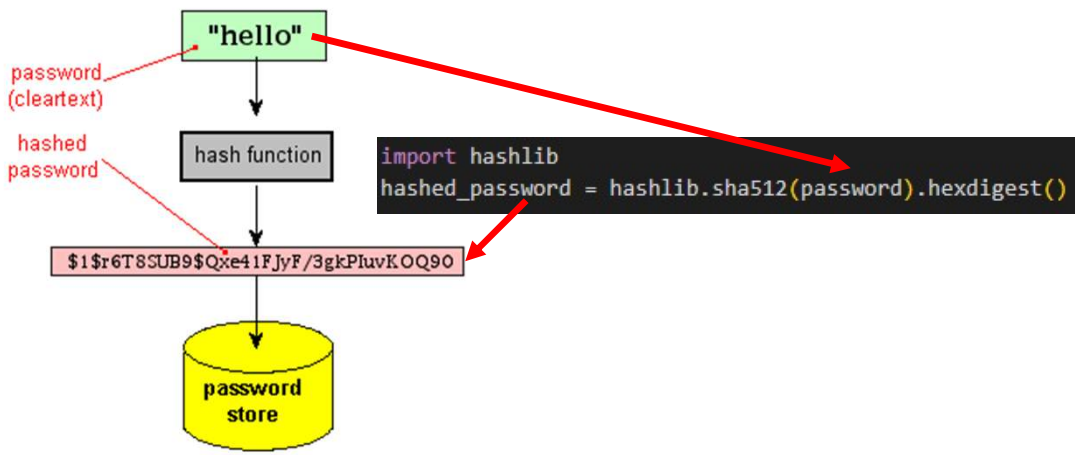
- Password Hashing
  - In most systems passwords are stored in a protected (i.e., hash) form.
  - Snooper that gains internal access to the system cannot easily retrieve/steal passwords.
- Login Process
  1. Every time a user logs in, password handling software runs the hash algorithm against the submitted password value.
  2. If (computed hash value === stored hash value) ➔ access is granted.



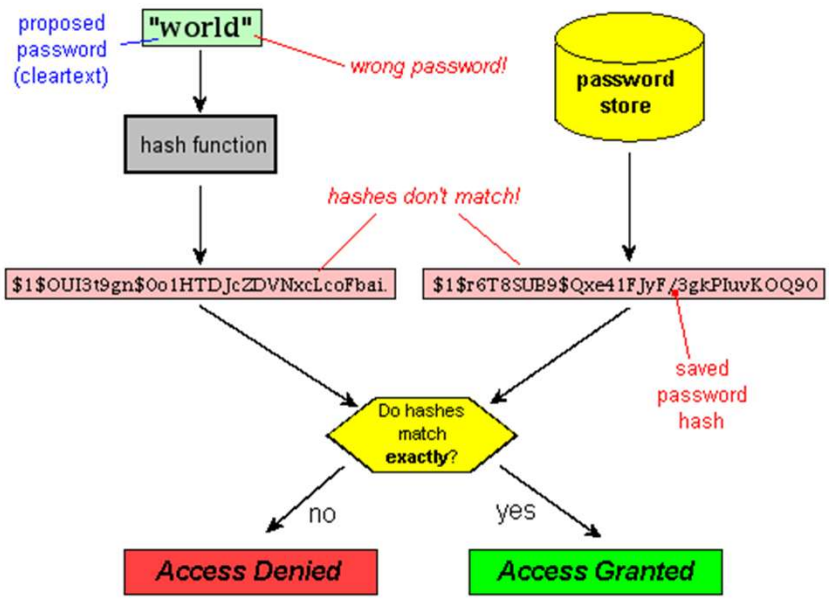
Some companies don't hash passwords, meaning that if a cracker gets into their system they can steal them. This is particularly bad when a user has the same password for multiple sites!

Password Hashing

- Password Hashing: Storage and Authentication



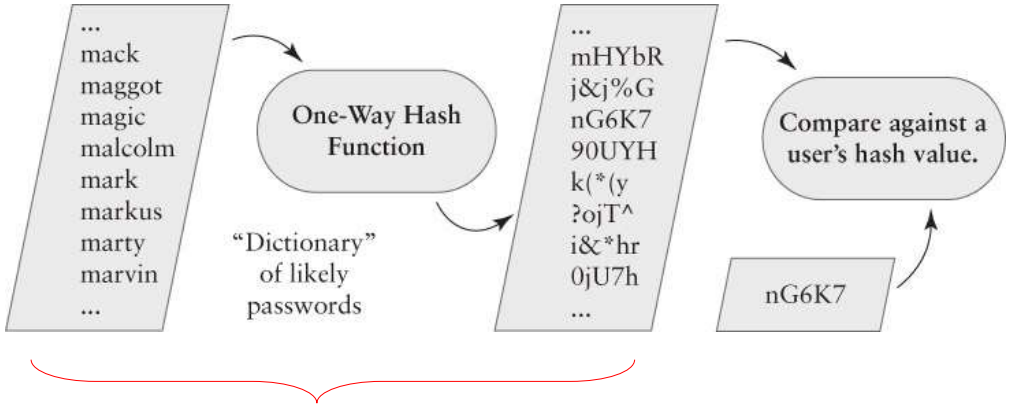
Storing hash instead of password



Testing a password against stored hash

Password Salting

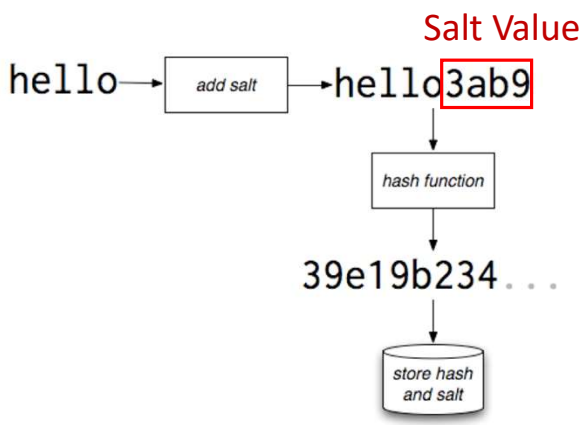
- Dictionary Attack in Password Cracking
  - Assuming: the threat actor got hold of our users' signing credentials stored in the database. Of course, the stored password are all hashed, so she needs to find out what are the corresponding passwords to the stored hashed values [this is known as offline password cracking!]
  - is a type of cyber attack that involves attempting to guess a password or encryption key by systematically trying every word in a dictionary or a list of commonly used passwords. [this is better than brute force in many cases, why? In class discussion]



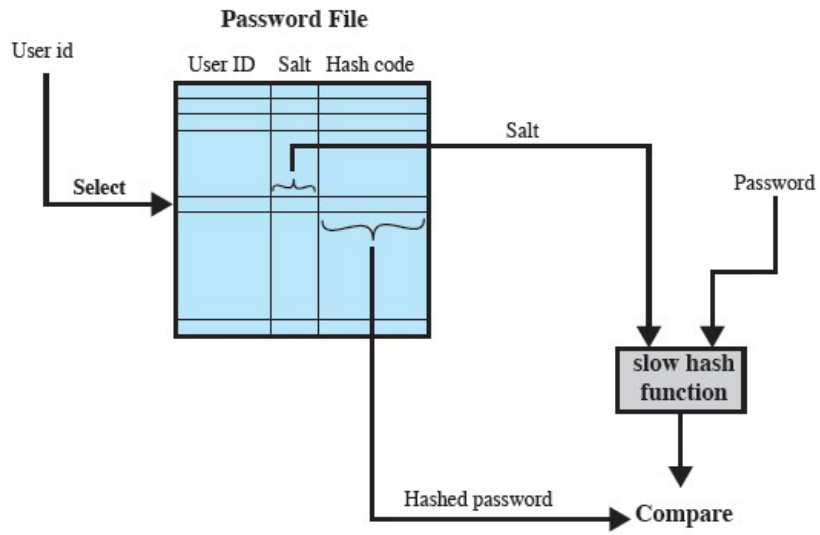
This part of computation is more expensive and time consuming

Password Salting – cont'd

- To make a dictionary-based password cracking to go slower – we can use salting.
- Adding a unique random value to each password before hashing:
  - Both the hash and salt are stored.
  - Does not fully prevent against password cracking but render pre-computed password hash lists useless! **[Why? In-class discussion]**




```
import hashlib, uuid
salt = uuid.uuid4().hex
hashed_password = hashlib.sha512(password + salt).hexdigest()
```



(b) Verifying a password

Password Salting

- Benefits – in case of a compromised Password File/Database
  - Render precomputed dictionaries useless!
  - Prevents duplicate passwords from being visible in password file!
  - Becomes impossible to find out whether a person has used the same password on multiple systems.

Password in plaintext	Salt Value			Password's Hash Digest
"adminadmin" +	"fdfhj"	→	Hash Function →	"asdhui@#$djfhhkxzxcz"
"adminadmin" +	"gwdk"	→	Hash Function →	"34asd23yrnwc@@asdf"

Same password

Different salt value

Cryptographic Key Derivation Function (KDF)

- Is a mathematical algorithm that takes an input (such as a password or passphrase) and generates one or more secret cryptographic keys, which can be used to encrypt or decrypt data.
  - The KDF uses the input to generate keys that are difficult for an attacker to guess, even if they know the input.
  - KDFs are commonly used in password-based encryption, where a user's password is used to derive a key for encrypting their data.
  - Also good as a proxy to password hashing for authentication – but not widely exercised.

```
>>> import os
>>> from cryptography.hazmat.primitives import hashes
>>> from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC
>>> # Salts should be randomly generated
>>> salt = os.urandom(16)
>>> # derive
>>> kdf = PBKDF2HMAC(
...     algorithm=hashes.SHA256(),
...     length=32,
...     salt=salt,
...     iterations=480000,
... )
>>> key = kdf.derive(b"my great password")
>>> # verify
>>> kdf = PBKDF2HMAC(
...     algorithm=hashes.SHA256(),
...     length=32,
...     salt=salt,
...     iterations=480000,
... )
>>> kdf.verify(b"my great password", key)
```

https://cryptography.io/en/latest/hazmat/primitives/key-derivation-functions/

Encrypted Communication Channel

- Always use TLS to establish network connections
  - TLS provides both encryption and authentication. You need to authenticate TLS certifications to ensure their validity.
    - Threat actors could forge TLS certificates!

```
import socket
import ssl

# SET VARIABLES
packet, reply = "<packet>SOME_DATA</packet>", ""
HOST, PORT = 'XX.XX.XX.XX', 4434

# CREATE SOCKET
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.settimeout(10)

# WRAP SOCKET
wrappedSocket = ssl.wrap_socket(sock, ssl_version=ssl.PROTOCOL_TLSv1,
                                ciphers="ADH-AES256-SHA")

# CONNECT AND PRINT REPLY
wrappedSocket.connect((HOST, PORT))
wrappedSocket.send(packet)
print wrappedSocket.recv(1280)

# CLOSE SOCKET CONNECTION
wrappedSocket.close()
```

Encrypted Communication Channel – Cont'd

- Certificate Pinning (CP): is a security technique that involves associating a particular digital certificate with a specific domain or endpoint, and then only accepting that certificate for subsequent connections to that domain or endpoint.
  - This means that even if an attacker were to present a valid certificate from a trusted Certificate Authority (CA), the connection would be rejected if it did not match the pinned certificate.

```python
import ssl, socket, pprint, os, sys

# Prepare context, including reference to the server's certificate
context = ssl.SSLContext(ssl.PROTOCOL_TLS)
context.verify_mode = ssl.CERT_REQUIRED
context.check_hostname = False
context.load_verify_locations(cafile='server_cert.pem')
cnx = context.wrap_socket(socket.socket(socket.AF_INET))

# Connect and evaluate
cnx.connect(('www.hostnametoconnect.com', 8888))
```

  - Certificate pinning is a good security practice because it provides an additional layer of protection against man-in-the-middle (MITM) attacks, where an attacker intercepts and alters communications between two parties. Without certificate pinning, an attacker with a valid certificate could intercept the connection and decrypt the data being transmitted without being detected.
  - certificate pinning requires careful management and updating of pinned certificates, as any changes to the certificate (such as a renewal or revocation) may break the pinned connection. Additionally, it may create some operational challenges when managing a large number of endpoints or domains, as each one requires specific certificates to be pinned.

Be Aware …

- Not all cryptographic schemes in-use today are secure!
  - Symmetric Ciphers
    - DES is not secure, use 3DES instead if you must! But 3DES will not be secure in the near future!
    - Use AES when you can!
  - Asymmetric Ciphers
    - All our asymmetric ciphers are secure for the time being. But they can be easily broken with a large-scale Quantum Computer. Post quantum security is something important to consider in your organization!
  - Cryptographic Hashing Functions
    - Not all cryptographic hash functions are secure: MD2, MD4, MD5, SHA-1
    - Use SHA-256 or SHA-512 instead.